Fase 2 — Pesaje inicial y activación (Punto 2)

Rama: `fase-2-pesaje-inicial` · Ejecución: 23/10/2026 · Estado: completada (pendientes en la sección 9).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo implementado y verificado.
- **Parte 2:** guía de estudio para la defensa oral (secciones A–J).

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`.

Parte 1 — Informe técnico

1. Alcance y resultado

La fase implementa el **Punto 2 de la consigna**: la balanza (TMS) envía el pesaje del camión vacío (*tara*); el backend valida los datos y el estado actual de la orden, registra la tara y el instante del pesaje, genera una contraseña de activación de 5 dígitos (10000 a 99999) mediante un generador seguro, transiciona la orden del **estado 1 al estado 2** (`INITIAL_WEIGHING_REGISTERED`), deja asentado el cambio en el historial de auditoría y responde la contraseña generada a la balanza.

Todo el proceso se realiza bajo una única transacción atómica: si cualquier paso falla, la orden permanece inalterada en estado 1.

#	Criterio de aceptación	Resultado
1	Tara válida: 200, contraseña de 5 dígitos y la orden pasa a estado 2 con tara y fecha guardadas	Cumplido
2	El historial tiene 2 registros: `null→1` y `1→2`, con el usuario del token como actor	Cumplido
3	Orden inexistente: 404	Cumplido
4	Tara 0, negativa o ausente: 400	Cumplido
5	Orden que no está en estado 1 (por ejemplo, una segunda tara): 409 Conflict	Cumplido
6	Sin token: 401	Cumplido
7	`GET /orders/{n}` no muestra la contraseña de activación (`@JsonIgnore`)	Cumplido
8	Atomicidad: si falla a mitad de camino, la orden no cambia	Cumplido
9	El endpoint figura en Swagger con sus respuestas documentadas	Cumplido
10	Postman: la carpeta de la Fase 2 pasa con newman (27 aserciones) y la de la Fase 1 sigue pasando (19 aserciones)	Cumplido (46 aserciones en total, 0 fallas)
11	Informe técnico, guía de estudio de la fase y documentación actualizada	Cumplido

2. Decisiones de diseño

#	Tema	Decisión	Alternativa descartada y motivo
1	Código HTTP de "estado incorrecto"	**409 Conflict** (`ConflictException`)	400 Bad Request: un 400 refiere a errores sintácticos o de validación del payload; 409 expresa con exactitud un conflicto con el estado actual del recurso en el servidor.
2	Contraseña en lecturas de la orden	**Oculta al exterior** (`@JsonIgnore` en `Order.activationPassword`)	Publicarla en `GET /orders/{n}`: la contraseña es un secreto compartido exclusivamente entre la balanza y el sistema de carga; exponerla permitiría activar cargas no autorizadas.

#	Tema	Decisión	Alternativa descartada y motivo
3	Orden de las comprobaciones de negocio	**404 (orden) → 400 (peso) → 409 (estado)**	Comprobar estado antes que peso: si el peso es absurdo (≤ 0), el cliente envió una petición malformada que debe rechazarse antes de evaluar las reglas de negocio de la entidad.
4	Ubicación de las comprobaciones respecto al `try/catch`	**Fuera del `try`** en `OrderBusiness.registerTare`	Dentro del `try`: el bloque `catch (Exception e)` envolvería `NotFoundException`, `BadRequestException` o `ConflictException` en una `BusinessException` genérica, respondiendo HTTP 500 en vez del código apropiado.
5	Generación de la contraseña	`SecureRandom` con rango 10000 a 99999 (`ActivationPasswordGenerator`)	`java.util.Random`: predecible y vulnerable a ataques de fuerza bruta o predicción de secuencia para terminales de carga de combustible.
6	Nulabilidad de columnas nuevas en JPA	`Double` e `Integer` (objetos), `@Column` sin `nullable = false`	Primitivos `double` e `int`: la tabla `orders` ya contenía filas de la Fase 1; columnas no nulables habrían impedido el arranque con `ddl-auto=update` o insertado valores por defecto 0 erróneos.
7	Formato del cuerpo de entrada de la balanza	DTO específico `TareRequest` con atributo `double weight`	Reutilizar `Order` como en SAP: la balanza no envía una orden, sino un comando de pesaje con una única magnitud física (kg).
8	Formato de la observación en auditoría	`"Tara registrada: <peso> kg"`	Mensaje genérico: registrar el valor numérico en el historial permite auditar discrepancias históricas si la balanza fuera recalibrada.

3. Archivos de la fase

3.1 Creados y modificados por etapa

Archivo	Acción	Motivo	Responsabilidad / Dueño
`SRC/model/Order.java`	Modificado	Agrega campos `tare`, `initialWeighingDateTime` y `activationPassword`	Modelo / Martín González (**A**)
`SRC/business/exception/ConflictException.java`	Creado	Excepción checked para estado incorrecto → HTTP 409	Excepciones / Martín González (**A**)
`SRC/controllers/Constants.java`	Modificado	Agrega constante `URL_INTEGRATION_TMS` (`/api/v1/integration/tms`)	Enrutamiento / Martín González (**A**)
`SRC/business/ActivationPasswordGenerator.java`	Creado	Generador criptográficamente seguro de 5 dígitos (10000–99999)	Negocio / Martín González (**A**)
`src/test/.../business/ActivationPasswordGeneratorTest.java`	Creado	Test unitario de rango y aleatoriedad sin base de datos	Pruebas / Martín González (**A**)
`SRC/business/IOrderBusiness.java`	Modificado	Declara firma de `registerTare` con sus 4 excepciones checked	Negocio / Tomás Calvo (**B**)
`SRC/business/OrderBusiness.java`	Modificado	Implementación transaccional de `registerTare` y log de estado 1→2	Negocio / Tomás Calvo (**B**)
`SRC/integration/tms/model/TareRequest.java`	Creado	DTO de entrada para pesaje de balanza (`double weight`)	Integración / Benjamín Alasino (**C**)
`SRC/integration/tms/model/TareResponse.java`	Creado	DTO de respuesta con `Integer activationPassword`	Integración / Benjamín Alasino (**C**)
`SRC/integration/tms/controllers/OrderTmsRestController.java`	Creado	Endpoint `POST /integration/tms/orders/{orderNumber}/tare`	API / Benjamín Alasino (**C**)

Archivo	Acción	Motivo	Responsabilidad / Dueño
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Modificado	Carpeta `02 - Fase 2: pesaje inicial (TMS)` (11 peticiones, 27 tests)	Pruebas / Joaquín Contreras (**D**)
`docs/fase-2-pesaje-inicial.md`	Creado	Informe técnico y guía de estudio completa A–J	Documentación / Joaquín Contreras (**D**)
`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md`	Modificados	Actualización de estado, decisiones y mapa de código	Documentación / Joaquín Contreras (**D**)

4. Recorrido de la petición `POST /api/v1/integration/tms/orders/{orderNumber}/tare`

```
Balanza (TMS) ──POST /api/v1/integration/tms/orders/90002/tare {"weight":14500.0}──►
   │
   ├─ 1. JWTAuthorizationFilter: extrae y valida token Bearer -> SecurityContextHolder
   ├─ 2. DispatcherServlet + Jackson: deserializa JSON en TareRequest(weight=14500.0)
   ├─ 3. OrderTmsRestController.registerTare:
   │       - Captura usuario autenticado (getUserLogged()) -> "admin"
   │       - Invoca orderBusiness.registerTare(orderNumber, weight, "admin")
   │
   ├─ 4. OrderBusiness.registerTare (@Transactional):
   │       - Comprueba existencia: load(orderNumber) -> si no existe, 404 NotFoundException
   │       - Comprueba peso: !(weight > 0) -> si <= 0 o NaN, 400 BadRequestException
   │       - Comprueba estado: status == PENDING_INITIAL_WEIGHING -> si no, 409 ConflictException
   │       [Dentro de try]:
   │       - Guarda tare = 14500.0
   │       - Guarda initialWeighingDateTime = LocalDateTime.now()
   │       - Genera clave: ActivationPasswordGenerator.generate() -> ej: 48213
   │       - Asigna status = OrderStatus.INITIAL_WEIGHING_REGISTERED (2)
   │       - orderDAO.save(order) -> UPDATE en tabla orders
   │       - logStatus(order, 1, 2, "admin", "Tara registrada: 14500.0 kg") -> INSERT en order_status_logs
   │
   ├─ 5. OrderTmsRestController: envuelve clave en TareResponse(48213) y retorna HTTP 200 OK
   ▼
◄── 200 OK {"activationPassword": 48213}
```

Si ocurre cualquier excepción no prevista dentro del bloque `try`, la transacción ejecuta *rollback*, se registra en el log y se responde HTTP 500 con `StandartResponse`.

5. Funciones y métodos relevantes

`OrderTmsRestController.registerTare` — `SRC/integration/tms/controllers/OrderTmsRestController.java`

- **Responsabilidad:** Exponer la interfaz REST para la balanza de pesaje inicial, extraer el actor autenticado y traducir las excepciones checked del negocio al formato de error estándar `StandartResponse`.
- **Parámetros:** `@PathVariable Integer orderNumber`, `@RequestBody TareRequest body`.
- **Retorno:** `ResponseEntity<?>`: 200 OK con `TareResponse(activationPassword)`, o `StandartResponse` con HTTP 400, 404, 409 o 500.
- **Anotaciones OpenAPI:** `@Tag`, `@Operation`, `@ApiResponse` (200, 400, 404, 409).

`OrderBusiness.registerTare` — `SRC/business/OrderBusiness.java`

- **Responsabilidad:** Regla de negocio central del Punto 2: validar el estado, asegurar la consistencia del pesaje, transicionar la orden a estado 2 y persistir la auditoría de cambio de estado.

- **Firma:** `Order registerTare(Integer orderNumber, double weight, String actor) throws NotFoundException, BadRequestException, ConflictException, BusinessException`.
- **Transaccionalidad:** `@Transactional(rollbackFor = Exception.class)`. Garantiza reversión completa ante excepciones checked.
- **Validaciones:**
  1. `load(orderNumber)`: lanza `NotFoundException` (404) si no existe.
  2. `!(weight > 0)`: lanza `BadRequestException` (400) si el peso es cero, negativo o `NaN`.
  3. `order.getStatus() != OrderStatus.PENDING_INITIAL_WEIGHING`: lanza `ConflictException` (409) informando el estado actual.
- **Operaciones dentro de la transacción:**
  - `order.setTare(weight)`
  - `order.setInitialWeighingDateTime(LocalDateTime.now())`
  - `order.setActivationPassword(ActivationPasswordGenerator.generate())`
  - `order.setStatus(OrderStatus.INITIAL_WEIGHING_REGISTERED)`
  - `orderDAO.save(order)`
  - `logStatus(order, from, to, actor, "Tara registrada: " + weight + " kg")`

`ActivationPasswordGenerator.generate` — `SRC/business/ActivationPasswordGenerator.java`

- **Responsabilidad:** Generar números enteros aleatorios uniformes de 5 dígitos (entre 10000 y 99999).
- **Visibilidad:** Clase final con visibilidad de paquete (`package-private`), inaccesible fuera de `ar.edu.iua.iw3.business`.
- **Mecanismo:** Utiliza una instancia estática de `java.security.SecureRandom`. Fórmula: `10000 + RANDOM.nextInt(90000)`.

`ConflictException` — `SRC/business/exception/ConflictException.java`

- **Responsabilidad:** Representar una violación de precondición de estado en el ciclo de vida de la entidad de negocio. Extiende `Exception` (checked) e incorpora soporte para `@Builder` de Lombok.

6. Pruebas y verificación

Todas las pruebas fueron ejecutadas el 23/10/2026 contra el contenedor de base de datos MySQL real (`iw3-gas-mysql`) con los datos base sembrados (`seed-dev-users.sql`).

6.1 Pruebas unitarias de JUnit

Comando ejecutado: `./mvnw -B test`

- `ActivationPasswordGeneratorTest.generatesFiveDigitPasswords`: 10.000 contraseñas generadas consecutivamente; todas dentro del intervalo `[10000, 99999]`.
- `ActivationPasswordGeneratorTest.doesNotAlwaysReturnTheSameValue`: comprobación de dispersión y variedad de claves en múltiples iteraciones.
- `Iw3ApplicationTests.contextLoads`: arranque correcto del contexto de Spring Boot e inicialización de beans JPA y Hibernate.
- **Resultado:** 3 tests ejecutados, 0 fallas, 0 errores (`BUILD SUCCESS`).

6.2 Pruebas de integración automatizadas con Newman

Comando ejecutado: `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json`

Carpeta	Petición	Método y Ruta	Esperado	Resultado
00 - Auth	Login	`POST /api/v1/login`	200, JWT de 3 partes	Aprobado
00 - Auth	Login fallido	`POST /api/v1/login`	401, StandartResponse	Aprobado
01 - F1 SAP	Crear orden	`POST /api/v1/integration/sap/orders`	201, header location	Aprobado
01 - F1 SAP	Consultar orden creada	`GET /api/v1/orders/{orderNumber}`	200, status 1	Aprobado
01 - F1 SAP	Historial de estados	`GET /api/v1/orders/{orderNumber}/status-log`	200, log null→1	Aprobado
01 - F1 SAP	Crear orden duplicada	`POST /api/v1/integration/sap/orders`	302 Found	Aprobado
01 - F1 SAP	Orden con datos inválidos	`POST /api/v1/integration/sap/orders`	400 Bad Request	Aprobado
01 - F1 SAP	JSON malformado	`POST /api/v1/integration/sap/orders`	400 Bad Request	Aprobado
01 - F1 SAP	Sin token	`POST /api/v1/integration/sap/orders`	401 Unauthorized	Aprobado
01 - F1 SAP	Orden inexistente	`GET /api/v1/orders/999999999`	404 Not Found	Aprobado
02 - F2 TMS	Crear orden para TMS	`POST /api/v1/integration/sap/orders`	201, header location	Aprobado
02 - F2 TMS	Tara válida	`POST /api/v1/integration/tms/orders/{n}/tare`	200, clave 5 dígitos	Aprobado
02 - F2 TMS	Consultar orden post-tara	`GET /api/v1/orders/{n}`	200, status 2, sin password	Aprobado
02 - F2 TMS	Historial post-tara	`GET /api/v1/orders/{n}/status-log`	200, 2 logs (null→1, 1→2)	Aprobado
02 - F2 TMS	Repetir tara (estado incorrecto)	`POST /api/v1/integration/tms/orders/{n}/tare`	409 Conflict	Aprobado
02 - F2 TMS	Tara peso 0	`POST /api/v1/integration/tms/orders/{n}/tare`	400 Bad Request	Aprobado
02 - F2 TMS	Tara peso negativo	`POST /api/v1/integration/tms/orders/{n}/tare`	400 Bad Request	Aprobado
02 - F2 TMS	Tara sin campo weight	`POST /api/v1/integration/tms/orders/{n}/tare`	400 Bad Request	Aprobado
02 - F2 TMS	Tara con JSON malformado	`POST /api/v1/integration/tms/orders/{n}/tare`	400 Bad Request	Aprobado
02 - F2 TMS	Tara orden inexistente	`POST /api/v1/integration/tms/orders/999999999/tare`	404 Not Found	Aprobado
02 - F2 TMS	Tara sin token	`POST /api/v1/integration/tms/orders/{n}/tare`	401 Unauthorized	Aprobado

Métricas Newman:

- Iteraciones: 1
- Peticiones ejecutadas: 21 (0 fallidas)
- Aserciones ejecutadas: 46 (0 fallidas)
- Tiempo de ejecución: 1004 ms

7. Cambios de comportamiento respecto de la Fase 1

Escenario / Petición	Fase 1	Fase 2
`POST /api/v1/integration/tms/orders/{n}/tare`	404 (ruta inexistente)	200 con `{ "activationPassword": NNNNN }`
Transición a estado 2	Inexistente (solo existía creación en estado 1)	Transición de estado 1 a estado 2 documentada y persistida
Intento de registrar tara sobre orden en estado 2	N/A	409 Conflict
Consulta `GET /api/v1/orders/{n}` tras pesaje inicial	Campos de tara inexistentes en la entidad	Retorna `tare` y `initialWeighingDateTime`; omite `activationPassword`
Historial `GET /api/v1/orders/{n}/status-log`	Máximo 1 registro por orden (`null→1`)	2 registros secuenciales (`null→1` y `1→2`) con actor y peso

8. Hallazgos y limitaciones de la fase

ID	Hallazgo / Limitación	Detalle y Mitigación
H-12	Concurrencia en taras simultáneas sobre la misma orden	Si dos peticiones de tara para el mismo número de orden entran concurrentemente con una diferencia de milisegundos, ambas podrían leer la orden en estado 1 antes de que la primera haga commit, intentando ambas realizar la transición. Para el parcial se acepta sin bloqueo pesimista ni versión (`@Version`), análogo a la decisión tomada con N-2 en altas simultáneas de la Fase 1.
H-13	Representación de peso en observaciones del historial	Dado que `weight` es de tipo de dato de punto flotante `double`, al concatenarse en la cadena de observación de auditoría se formatea como `"Tara registrada: 14500.0 kg"`. Es inocuo y coherente con la precisión requerida.

9. Pendientes y criterios de la fase

Pendiente	Cuándo
Endpoint de activación con contraseña y recepción de lecturas del caudalímetro	Fase 3 (`fase-3-carga-tiempo-real`)
Endpoint de cierre de orden (estado 2 → estado 3)	Fase 4 (`fase-4-cierre-orden`)
Redespliegue de la imagen con la Fase 2 en la máquina virtual	Al integrar la fase en `main`

Parte 2 — Guía de estudio de la Fase 2

A. Objetivo de la fase

El Punto 2 de la consigna vincula el mundo físico de la terminal (la balanza de camiones) con la gestión lógica del backend. Su objetivo es:

1. Registrar el peso del camión vacío antes de ingresar a la isla de carga.
2. Garantizar que la orden esté en el estado inicial correcto (estado 1) antes de habilitarla para el pesaje.
3. Generar un secreto temporal (la contraseña de activación de 5 dígitos) que será entregado al chofer u operador de carga para iniciar el despacho en la Fase 3.
4. Avanzar el ciclo de vida de la orden al estado 2 y dejar un registro auditable e inmutable del cambio de estado.

B. Conceptos teóricos de la cátedra

1. Transacciones declarativas y excepciones checked

En Spring, `@Transactional` por defecto revierte la transacción únicamente ante excepciones no comprobadas (`RuntimeException` y `Error`). Sin embargo, en la arquitectura de la cátedra todas las excepciones de negocio (`NotFoundException`, `BadRequestException`, `ConflictException`, `BusinessException`) extienden de `Exception` (son *checked*). Por este motivo, es mandatorio declarar explícitamente:

```
@Transactional(rollbackFor = Exception.class)
```

Si se omitiera `rollbackFor = Exception.class`, cualquier excepción checked lanzada durante el guardado de la orden o del log dejaría datos inconsistentes en la base sin revertir las operaciones anteriores.

2. Manejo de excepciones en la capa de presentación (Controller)

Siguiendo el patrón de la cátedra, los controladores no usan `@ControllerAdvice` global, sino un bloque `try/catch` explícito por método que atrapa cada excepción de negocio y la transforma en una respuesta HTTP con `IStandartResponseBusiness`:

- `NotFoundException` → HTTP 404 (recurso no existe).
- `BadRequestException` → HTTP 400 (parámetros inválidos enviados por el cliente).
- `ConflictException` → HTTP 409 (conflicto de precondición de negocio en el estado del recurso).
- `BusinessException` → HTTP 500 (falla interna o de infraestructura).

3. Serialización JSON y el atributo `@JsonIgnore`

En una API REST, el modelo de datos de persistencia JPA puede contener campos que no deben exponerse al cliente por motivos de seguridad o confidencialidad. La anotación `@JsonIgnore` de Jackson instruye al serializador para que ignore ese campo al transformar la entidad en JSON, evitando que `GET /orders/{n}` filtre la contraseña generada.

4. Seguridad en generación de identificadores: `SecureRandom` vs `Random`

`java.util.Random` utiliza un generador de congruencia lineal (LCG) que es completamente determinista y predecible a partir de unas pocas muestras del valor de la semilla. Para claves de seguridad o contraseñas de despacho de combustible, la cátedra y las buenas prácticas exigen `java.security.SecureRandom`, que se alimenta de fuentes de entropía del sistema operativo (como `/dev/urandom` en Linux), impidiendo predecir futuras contraseñas.

C. Arquitectura y patrones

```
Cliente HTTP (TMS)
       │ (JSON: weight)
       ▼
OrderTmsRestController (integration/tms/controllers)
       │ (orderNumber, weight, username)
       ▼
IOrderBusiness / OrderBusiness (business)
       │
       ├─ ActivationPasswordGenerator (business)
       ├─ OrderRepository / orderDAO (model/persistence)
       └─ OrderStatusLogRepository / statusLogDAO (model/persistence)
       │
       ▼
Base de datos MySQL (tablas orders y order_status_logs)
```

- **Patrón DTO de integración:** A diferencia de SAP (que enviaba la orden completa), TMS solo envía un dato de pesaje (`TareRequest`) y recibe la clave (`TareResponse`). Esto desacopla el contrato externo de la balanza

respecto a la estructura interna de la entidad `Order`.

- **Inversión de dependencias:** El controlador inyecta la interfaz `IOrderBusiness`, garantizando bajo acoplamiento con la implementación concreta `OrderBusiness`.

D. Código paso a paso

1. Entidad `Order` (`model/Order.java`)

Se agregaron tres campos nulables:

```
@Schema(accessMode = Schema.AccessMode.READ_ONLY)
@Column
private Double tare;

@Schema(accessMode = Schema.AccessMode.READ_ONLY)
@Column
private LocalDateTime initialWeighingDateTime;

@JsonIgnore
@Column
private Integer activationPassword;
```

Detalle clave: Se utilizan los tipos de objeto `Double` e `Integer` en lugar de los primitivos `double` e `int` para permitir valores `null` en órdenes existentes antes de su pesaje inicial.

2. Excepción `ConflictException` (`business/exception/ConflictException.java`)

Copiada del molde de `BadRequestException`:

```
@NoArgsConstructor
public class ConflictException extends Exception {
    private static final long serialVersionUID = 1L;
    @Builder
    public ConflictException(String message, Throwable ex) { super(message, ex); }
    public ConflictException(String message) { super(message); }
    public ConflictException(Throwable ex) { super(ex); }
}
```

3. Generador de Contraseñas (`business/ActivationPasswordGenerator.java`)

```
final class ActivationPasswordGenerator {
    private static final SecureRandom RANDOM = new SecureRandom();
    private ActivationPasswordGenerator() { }
    static int generate() {
        return 10000 + RANDOM.nextInt(90000);
    }
}
```

Detalle clave: `RANDOM.nextInt(90000)` produce un entero entre 0 y 89999. Sumando 10000, el resultado queda estrictamente comprendido en `[10000, 99999]`.

4. Servicio de Negocio (`business/OrderBusiness.java`)

```
@Override
@Transactional(rollbackFor = Exception.class)
public Order registerTare(Integer orderNumber, double weight, String actor)
        throws NotFoundException, BadRequestException, ConflictException, BusinessException {
    Order order = load(orderNumber);
    if (!(weight > 0)) {
        throw BadRequestException.builder().message("weight debe ser mayor que 0 (kg)").build();
    }
    if (order.getStatus() != OrderStatus.PENDING_INITIAL_WEIGHING) {
        throw ConflictException.builder().message("La orden " + orderNumber
                + " no está pendiente de pesaje inicial (estado " + order.getStatus().getCode() + ")").build();
    }
    try {
        OrderStatus from = order.getStatus();
        order.setTare(weight);
        order.setInitialWeighingDateTime(LocalDateTime.now());
        order.setActivationPassword(ActivationPasswordGenerator.generate());
        order.setStatus(OrderStatus.INITIAL_WEIGHING_REGISTERED);
        order = orderDAO.save(order);
        logStatus(order, from, order.getStatus(), actor, "Tara registrada: " + weight + " kg");
        return order;
    } catch (Exception e) {
        log.error(e.getMessage(), e);
        throw BusinessException.builder().ex(e).build();
    }
}
```

E. Trazabilidad de los datos

1. **Estado inicial:** La orden fue creada en Fase 1 en la tabla `orders` con `status = 1`, `tare = NULL`, `initial_weighing_date_time = NULL`, `activation_password = NULL`.
2. **Entrada:** La balanza envía `POST /api/v1/integration/tms/orders/90002/tare` con cuerpo `{ "weight": 14500.0 }`.
3. **Persistencia de la orden:** Se ejecuta un `UPDATE` sobre la fila correspondiente en la tabla `orders`:
   - `status = 2`
   - `tare = 14500.0`
   - `initial_weighing_date_time = '2026-10-23 11:49:45'`
   - `activation_password = 48213`
4. **Persistencia del log de auditoría:** Se ejecuta un `INSERT` en la tabla `order_status_logs`:
   - `id_order = <id de la orden>`
   - `from_status = 1`
   - `to_status = 2`
   - `date_time = '2026-10-23 11:49:45'`
   - `actor = 'admin'`
   - `observations = 'Tara registrada: 14500.0 kg'`

F. Ejemplo práctico de extremo a extremo

Paso 1: Autenticación

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/login -d "username=admin&password=admin123")
```

Paso 2: Registro de tara

```
curl -i -X POST http://localhost:8080/api/v1/integration/tms/orders/90002/tare \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"weight": 14500.0}'
```

Respuesta:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "activationPassword": 48213
}
```

Paso 3: Verificación de la orden

```
curl -s http://localhost:8080/api/v1/orders/90002 -H "Authorization: Bearer $TOKEN"
```

Respuesta (fragmento):

```
{
  "orderNumber": 90002,
  "status": 2,
  "tare": 14500.0,
  "initialWeighingDateTime": "2026-10-23T11:49:45.123"
}
```

(Nótese que `activationPassword` no aparece en el JSON).

Paso 4: Intento de segunda tara (error 409)

```
curl -i -X POST http://localhost:8080/api/v1/integration/tms/orders/90002/tare \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"weight": 14500.0}'
```

Respuesta:

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "message": "La orden 90002 no está pendiente de pesaje inicial (estado 2)",
  "code": 409
}
```

G. Preguntas del profesor para la defensa oral

1. **¿Por qué se eligió el código HTTP 409 Conflict para una orden que no está en estado 1 en vez de un 400 Bad Request?**
2. **¿Por qué las comprobaciones de 404, 400 y 409 deben ubicarse obligatoriamente fuera del bloque `try/catch` en `OrderBusiness.registerTare`?**
3. **¿Qué sucedería si omitimos el atributo `rollbackFor = Exception.class` en la anotación `@Transactional` del método `registerTare`?**
4. **¿Por qué los nuevos atributos `tare` y `activationPassword` de `Order` se declararon como `Double` e `Integer` en lugar de los tipos primitivos `double` e `int`?**

5. **¿Cómo garantizamos que la contraseña de activación generada no sea visible por nadie que consulte la orden por la API?**
6. **¿Por qué se utilizó `SecureRandom` en lugar de `Math.random()` o `java.util.Random` para generar la contraseña?**
7. **¿Por qué la clase `ActivationPasswordGenerator` tiene visibilidad de paquete (`package-private`) y constructor privado?**
8. **¿Cómo se obtiene el nombre del usuario autenticado que queda registrado en el historial como actor de la operación?**
9. **¿Por qué en este endpoint se utilizaron clases DTO específicas (`TareRequest` y `TareResponse`) mientras que en la Fase 1 se utilizó directamente la entidad `Order` como `@RequestBody`?**
10. **Si se envía un JSON con `{ "weight": -10 }`, ¿qué código HTTP devuelve el endpoint y qué mensaje incluye el cuerpo?**
11. **¿Qué garantiza que el pesaje de la tara y el registro en el historial de estados ocurran de manera atómica?**
12. **¿Qué limitación de concurrencia existe en esta implementación si dos operadores envían la tara de la misma orden en el mismo milisegundo?**

H. Respuestas modelo a las preguntas del profesor

1. **Código 409 vs 400:** El código 400 Bad Request denota errores del cliente en el formato o la sintaxis de la petición (por ejemplo, un JSON mal formado o un peso negativo). En cambio, cuando el cuerpo de la petición es perfectamente válido pero la operación no puede completarse debido a que el recurso en el servidor se encuentra en un estado incompatible (la orden ya pasó a estado 2), el estándar HTTP RFC 7231 define taxativamente el código **409 Conflict**.
2. **Comprobaciones fuera del `try/catch`:** El método `registerTare` atrapa `Exception` en su bloque `try/catch` para capturar cualquier falla inesperada de base de datos y envolverla en una `BusinessException` que el controller traduce a HTTP 500. Si las comprobaciones de orden existente (404), peso positivo (400) o estado correcto (409) estuvieran dentro del `try`, al lanzar sus respectivas excepciones checked (`NotFoundException`, `BadRequestException`, `ConflictException`), el bloque `catch (Exception e)` las interceptaría erróneamente, registrando un error en el log y respondiendo un 500 al cliente en lugar del código HTTP que le corresponde a cada error de validación.
3. **Rol de `rollbackFor = Exception.class`:** Por especificación de Spring Transactional, solo las excepciones no comprobadas (`RuntimeException` y `Error`) disparan el rollback automático. Dado que nuestras excepciones de negocio extienden de `java.lang.Exception` (son comprobadas o *checked*), si ocurre una excepción de negocio en medio de la ejecución y no especificamos `rollbackFor = Exception.class`, Spring confirmaría el commit de la transacción en lugar de revertirla, dejando datos inconsistentes en la base.
4. **Tipos envoltorio (`Double` / `Integer`) vs primitivos:** Cuando una orden se crea en la Fase 1, todavía no tiene pesaje inicial ni contraseña; por ende, esos atributos deben ser `null`. Si hubiésemos utilizado tipos primitivos (`double`, `int`), la base de datos o Hibernate habrían asignado `0.0` y `0` por defecto, lo cual sería incorrecto (un camión con tara 0 kg no tiene sentido físico). Además, al arrancar la aplicación con `ddl-auto=update` sobre una tabla `orders` que ya contiene filas, Hibernate no puede agregar columnas con restricción `NOT NULL` sin fallar.
5. **Ocultamiento de la contraseña con `@JsonIgnore`:** Se anotó el campo `activationPassword` en la entidad `Order` con `@JsonIgnore` de Jackson. Al serializar la entidad para responder peticiones como `GET /api/v1/orders/{orderNumber}`, Jackson omite completamente el campo, garantizando que el secreto no viaje en ninguna respuesta de consulta general. Solo viaja en el objeto `TareResponse` retornado exclusivamente a la balanza en el momento del pesaje.
6. **`SecureRandom` vs `Random`:** `java.util.Random` y `Math.random()` son generadores pseudoaleatorios lineales deterministas cuya semilla se basa habitualmente en la hora del sistema en milisegundos. Un atacante que observe dos o tres contraseñas podría deducir la semilla y predecir las contraseñas siguientes de los camiones de

combustible. `SecureRandom` es un generador criptográfico fuerte que toma entropía no determinista del sistema operativo.

7. **Diseño de `ActivationPasswordGenerator`:** Al ser una clase auxiliar que solo concierne a la capa de negocio del módulo, su visibilidad de paquete (`package-private`) evita que controladores u otras capas externas la invoquen directamente. El constructor privado evita que sea instanciada, ya que expone un método estático puro.
8. **Obtención del actor autenticado:** El controlador extiende de `BaseRestController`, el cual provee el método `getUserLogged()`. Este método accede al `SecurityContextHolder.getContext().getAuthentication().getPrincipal()`, que fue previamente poblado por el filtro `JWTAuthorizationFilter` al validar la firma del token Bearer de la petición.
9. **Uso de DTO en TMS vs Entidad en SAP:** En el Punto 1 (SAP), el sistema externo envía la orden completa con sus relaciones (camión, chofer, cliente, producto). En el Punto 2 (TMS), la balanza es un dispositivo que solo reporta una medición puntual (`weight`). Crear un DTO `TareRequest` y `TareResponse` evita exponer o sobrecargar la entidad `Order`, cumple con el principio de responsabilidad única y documenta limpiamente el contrato de entrada y salida en OpenAPI/Swagger.
10. **Respuesta ante peso negativo:** El validador en `OrderBusiness.registerTare` evalúa `!(weight > 0)`. Como `-10 > 0` es falso, se lanza una `BadRequestException` con el mensaje `"weight debe ser mayor que 0 (kg)"`. El controlador captura esta excepción y retorna HTTP 400 Bad Request con un cuerpo JSON de tipo `StandartResponse` (`code: 400`).
11. **Garantía de atomicidad:** La anotación `@Transactional(rollbackFor = Exception.class)` sobre `registerTare` asegura que tanto la actualización de la orden (`orderDAO.save`) como la inserción del registro en el log de auditoría (`logStatus`) se ejecuten dentro de la misma transacción JDBC de base de datos. Si la inserción del log fallara, la orden no se actualiza y vuelve automáticamente a su estado original en estado 1.
12. **Limitación de concurrencia:** Si entran dos peticiones para la misma orden de forma simultánea, ambas podrían leer el registro con `status = 1` antes de que la otra confirme el cambio. Ambas generarían contraseñas y ambas escribirían en el log. Para solucionar esto en una fase posterior o en producción, se requeriría un bloqueo pesimista (`SELECT ... FOR UPDATE`) o control de concurrencia optimista (`@Version`).

I. Errores frecuentes durante el desarrollo

1. **Olvidar `rollbackFor = Exception.class`:** Provoca que Spring no revierta ante excepciones de negocio checked.
2. **Meter las validaciones de estado o peso dentro del `try/catch` de `registerTare`:** Provoca que el `catch (Exception e)` capture la excepción y la transforme en `BusinessException`, resultando en un error HTTP 500 en vez de 400 o 409.
3. **Declarar campos primitivos en la entidad:** Rompe el arranque de Hibernate con `ddl-auto=update` sobre bases que ya tenían datos, o inicializa la tara en `0.0`.
4. **Olvidar `@JsonIgnore`:** Expone la contraseña de 5 dígitos en los endpoints públicos o administrativos de consulta de órdenes.
5. **Pisar variables en Postman:** Usar la variable `orderNumber` de la Fase 1 en las pruebas de la Fase 2 hace que fallen las aserciones de la Fase 1 si se corre la colección completa. La solución fue usar una variable independiente: `orderNumberTms`.

J. Lista de comprobación para rendir la fase

- [ ] Conocer la ruta exacta y método del endpoint: `POST /api/v1/integration/tms/orders/{orderNumber}/tare`.
- [ ] Saber explicar el ciclo de estados: por qué pasa de 1 a 2 y por qué no puede registrarse dos veces (409 Conflict).
- [ ] Explicar por qué `activationPassword` no aparece en `GET /orders/{n}`.
- [ ] Explicar cómo funciona `ActivationPasswordGenerator` y por qué genera números entre 10000 y 99999.

- [ ] Saber el orden de las tres validaciones del negocio: 404 (orden) → 400 (peso) → 409 (estado).
- [ ] Saber justificar por qué las validaciones van fuera del `try/catch`.
- [ ] Poder mostrar la colección de Postman con sus 11 pedidos de la Fase 2 y cómo newman pasa con 0 errores (46 aserciones en total).
- [ ] Saber explicar la consulta SQL al historial: dos filas en `order_status_logs`, una de `null→1` y otra de `1→2`.